최기환

Frontend Engineer

rlghks3004@gmail.com GitHub gihwan-dev.tistory.com

---

01 **소프트웨어가 계속해서 나아질 수 있는 환경을 설계합니다.** 새로운 제품뿐만 아니라 기존 레거시 제품에도 테스트 코드를 도입하고 운영합니다. 코드 수정이 당장의 버그 해결에 그치지 않고 실질적인 개선으로 이어졌는지 확인할 수 있도록 합니다.

02 **좋은 개발자는 회사에 돈을 잘 벌어주는 개발자라고 생각합니다.** 항상 내가 하는 업무의 단기적, 장기적 가치와 필요한 리소스를 저울질하며 올바른 선택을 하려고 노력합니다.

03 **귀찮은 일을 가장 싫어합니다.** 반복되는 작업과 불필요한 일을 최대한 줄이려고 노력합니다.

경력

주식회사엑셈 / 프론트엔드 개발자 2024.11 - 현재

MaxGauge Dashboard: 통합 인스턴스 모니터링 입사 초기부터 담당

`React / TypeScript / TanStack Query`

여러 DB 인스턴스의 상태와 알람을 한 화면에서 확인하는 통합 모니터링 대시보드입니다. 입사 초기부터 기존 Vue Dashboard 6.0의 기획과 디자인, 개발을 전담했습니다.

- **고객사별 맞춤 개발에서 공통 기능 조합으로 전환**
  - 고객사별 커스텀 기능을 플래그로 구분해 코드를 수정할 때마다 다른 고객사 기능까지 점검해야 했습니다.
  - 모든 고객사에 같은 제품을 제공하되 필요한 기능만 켜서 납품하는 방향으로 정했습니다.
  - 기능 요청은 공통 제품에 반영하고 고객사별 설정에서 사용할 기능을 선택하도록 구현했습니다.
- **사내 디자인 시스템을 활용하도록 React로 전환**
  - 별도 기획자와 디자이너 없이 화면을 만들면서 작업마다 디자인과 테마가 달라졌습니다.
  - 사내 디자인 시스템이 React 기반이어서 이를 활용할 수 있도록 Vue 대시보드를 React로 전환했습니다.
  - 이후 공통 컴포넌트와 디자인 토큰을 기준으로 화면을 구현했습니다.
- **API 변경을 고려한 데이터 조회 구조 정비**
  - Vue 대시보드는 전역 변수에 강하게 의존했습니다. 전역 상태에 데이터를 공급하는 API도 전면 변경될 예정이었습니다.
  - React 전환 후에는 알람과 인스턴스 데이터의 조회와 변환을 서비스별 훅으로 옮기고 해당 전역 스토어를 제거했습니다.
  - 사용자 조작에 따른 화면 결과를 확인하는 테스트를 작성해 구조를 바꾼 뒤에도 기존 동작을 검증했습니다.

MaxGauge 차세대: DB 성능 분석

`React / TypeScript / TanStack Query / ECharts`

SQL과 세션, DB 객체의 정보를 분석하는 제품입니다. 초기에는 위젯 빌더와 SQL Detail, Object Detail의 화면 개발과 API 연동을 맡았습니다. 현재는 제품 전반의 기능 수정과 공통 컴포넌트 개발을 담당합니다.

- **위젯 빌더의 차트와 테이블 미리보기 개발**
  - 차트 미리보기의 데이터 준비와 표시 코드가 섞여 테이블 미리보기를 추가하기 어려웠습니다.
  - 데이터 준비와 화면 표시를 분리하고 시각화 종류별 렌더러를 두었습니다.
  - 같은 설정 흐름에서 차트와 테이블 미리보기를 제공하도록 바꿨습니다.
- **22개 화면의 테이블 동작과 컬럼 정의 통일**
  - 화면별 테이블 구현이 달라 정렬과 필터 동작, 컬럼 이름과 단위가 제각각이었습니다.
  - 범용 동작은 엔진에, 제품의 컬럼과 상세 화면 연결 규칙은 어댑터에 두고 화면에서는 컬럼과 데이터를 선언하도록 나눴습니다.
  - 필요한 엔진 수정과 22개 화면 전환을 직접 맡아 테이블 동작을 통일했습니다.
  - 259개 컬럼 정의를 한곳에서 관리해 이름과 타입을 화면마다 수정할 필요를 없앴습니다.
- **차트에서 선택한 SQL의 페이지부터 조회**
  - 차트에서 고른 SQL이 목록 뒤쪽에 있으면 앞 페이지를 모두 받은 뒤에야 해당 행을 표시했습니다.
  - 서버가 알려주는 목표 페이지를 먼저 받고 앞뒤 데이터는 스크롤할 때 조회하도록 바꿨습니다.
  - 조회와 캐시는 화면에서 관리하고, 엔진에는 미조회 구간의 가상 공간과 행 이동 기능을 추가했습니다.
  - 실데이터 20,212건의 한 사례에서 요청은 178건에서 2건으로, 선택 행 표시 시간은 212.5초에서 11.8초로 줄었습니다. (당시 전후 각 1회 비교)
- **차트 선명도를 해치던 리사이즈 최적화 제거**
  - 리사이즈 중 캔버스를 늘리고 200ms 뒤 다시 그리는 처리 때문에 글자와 점이 찌그러졌습니다.
  - CPU 4배 감속 조건에서 21만 점 Scatter의 끊김을 확인했습니다.
  - 출시 전 드문 대용량 조건에 맞추기보다 일반적인 화면의 선명도를 우선하기로 했습니다.
  - 캔버스 확대와 지연 처리를 제거하고 requestAnimationFrame으로 프레임당 한 번 실제 크기에 맞춰 다시 그리도록 바꿨습니다.
- **화면 규약을 AI 작업에 적용하는 스킬 작성**
  - 리뷰에서 반복 확인하던 화면 구성과 공통 컴포넌트 사용 규칙을 스킬로 정리했습니다.
  - AI가 화면을 만들거나 수정할 때 같은 규약과 참조 구현을 확인하도록 했습니다.
  - 스킬 도입 후 해당 규약에 대한 리뷰 지적은 더 이상 나오지 않았습니다.
- **변경 전후의 동작을 영상으로 비교하는 리뷰 도입**
  - 스크롤과 드래그처럼 움직임이 중요한 변경은 스크린샷만으로 차이를 판단하기 어려웠습니다.
  - 같은 조작을 변경 전후에 자동 녹화해 비교 영상으로 만드는 스킬을 작성했습니다.
  - 리뷰어가 직접 재현하지 않고 영상으로 동작 차이를 확인할 수 있게 했습니다.

MaxGauge 차세대 (계속)

- **공유 테스트 서버 밖에서 검증할 수 있는 실행 환경 구성**
  - 공유 테스트 서버는 데이터가 부족하거나 다른 팀이 함께 사용해 필요한 빌드를 올리고 부하를 주기 어려웠습니다.
  - 백엔드 서비스와 DB, 수집 데몬을 구성하고 기동과 로그인을 확인하는 절차를 스크립트와 스킬로 만들었습니다.
  - 원격 Linux에서 기동과 로그인, DB 접속을 확인해 별도 환경에서도 제품을 검증할 수 있게 했습니다.

- **CI 실행 시간을 측정하고 파이프라인 단축**
  - 테스트 증가로 CI가 느려졌지만 큐 대기 중앙값은 1.6초였습니다. 러너 분리보다 시간이 드는 실행 단계부터 살펴봤습니다.
  - Storybook을 미리 빌드해 제공하고 테스트별 실행 환경을 나누는 등 불필요한 준비와 중복 실행을 줄였습니다.
  - 당시 파이프라인 시간 중앙값은 198초에서 94초로 줄었습니다. (변경 전 20회, 후 4회 비교, 재시도와 캐시 재생성 제외)

exem-table: 사내 React 테이블 라이브러리 `2025.06경 인수 - 현재`

`React / TypeScript / TanStack Table / Vitest`

MaxGauge 차세대와 통합 대시보드에서 사용하는 React 테이블 라이브러리입니다. 차세대 프로젝트 안에 있던 컴포넌트를 인수해 개발과 유지보수를 맡고 있습니다.

- **기능별 API와 상태 관리 주체 정리**
  - 기능마다 API가 달랐고 사용하는 화면과 테이블 내부가 같은 상태를 각자 관리해 동기화가 복잡했습니다.
  - 옵션을 기능별로 묶고, 화면이 상태를 관리하는 제어 방식과 엔진이 관리하는 비제어 방식을 구분했습니다.
  - 중간 상태 계층을 제거해 사용 제품과 테이블 엔진 사이의 상태 동기화 단계를 줄였습니다.

- **가상화를 기본으로 하는 렌더링 구조로 전환**
  - 기존 table 태그 기반 구현에서는 컬럼 리사이즈나 이동 UI를 추가할 때 DOM 배치에 제약이 있었습니다.
  - 가상화를 옵션으로 두면 같은 기능을 두 경로로 구현하고 검증해야 했습니다. 가상화의 제약보다 두 경로를 유지하는 부담이 크다고 판단했습니다.
  - div 기반으로 전환하고 가상화를 기본으로 적용해 컬럼 고정과 행 확장도 같은 구조에서 처리했습니다.

- **내부 구현 대신 사용자 동작을 확인하는 테스트 작성**
  - 렌더링 구조를 바꾸자 내부 구현에 의존하던 기존 테스트도 깨졌습니다.
  - 공개 API로 테이블을 렌더링하고 사용자 조작에 따른 DOM 결과를 확인하도록 통합 테스트를 다시 작성했습니다.
  - 이후 실제 브라우저에서 조작 결과를 검증하는 테스트를 추가해 엔진 전환 때도 기존 동작을 확인했습니다.

- **TanStack Table v9 전환과 상태 구독 범위 축소**
  - 최상위 컴포넌트가 전체 상태를 구독해 상태 하나가 바뀌어도 테이블 전반이 렌더링에 참여했습니다.
  - 엔진 의존 코드를 분리한 뒤 기능별로 v9로 이전했습니다.
  - 각 컴포넌트가 필요한 상태만 구독하도록 바꿨습니다.
  - v9 전환 후 10,000행, 8컬럼에서 14자 검색어 입력의 누적 React 렌더 시간이 1,927ms에서 61ms로 줄었습니다. (당시 동일 조건 비교, 5회 중앙값)
  - 차세대의 테이블 사용 코드도 새 API로 이전하면서 타입 검사로 잡히지 않던 커스텀 정렬 연결을 4개 화면에서 수정했습니다.

MaxGauge 기존 제품군: 고객 요청부터 패키징까지

2026.09부터 전담

`JavaScript / ExtJS / Playwright / Claude Code / Codex`

DB 성능 모니터링 제품 MFO, MFT, MFA, MFD, MFS의 프론트엔드 유지보수를 전담합니다. 고객 요청 분석부터 재현, 구현, 검증, 버전 관리와 패키징까지 맡습니다. 대시보드와 차세대, exem-table 개발도 병행합니다.

- **제품 규칙을 정리해 AI가 유지보수에 참고하도록 구성**
  - 기존 처리 규칙은 Logic map에, 화면과 API의 관계는 Spec map에 정리했습니다.
  - 유지보수할 때 변경 의도를 전달하면 AI가 이 문서를 참고해 구현과 검증을 진행하도록 했습니다.
- **반복된 버그를 리뷰 스킬과 회귀 테스트에 반영**
  - 과거 MR에서 반복된 버그와 사이드 이펙트를 조사해 AI 리뷰 스킬의 점검 항목에 반영했습니다.
  - 변경 커밋의 리뷰 기록이 있어야 MR 작성과 패키지 전달 절차를 진행할 수 있도록 했습니다.
  - 수정한 동작을 테스트 코드로 남겨 후속 변경에서도 같은 문제가 다시 생기는지 확인했습니다.
- **시나리오 이미지로 동작을 확인하는 MR 리뷰**
  - 변경 시나리오별로 수정 전후 화면을 MR에 첨부하도록 했습니다.
  - ExTuner 연동 MR은 AI 코드 리뷰와 테스트를 거친 뒤, 주로 시나리오 이미지로 동작을 확인하고 병합했습니다.
- **버전에 포함된 작업과 소스 추적**
  - 기존에는 어떤 버전에 어떤 작업이 포함됐는지 추적하기 어려웠습니다.
  - 작업 MR과 Client 및 SQL 커밋, 태그, 패키지 기록을 연결해 버전에 포함된 작업과 배포 소스를 확인할 수 있게 했습니다.
- **MFO에서 ExTuner 튜닝 요청과 결과 확인**
  - MFO에서 SQL 튜닝 도구 ExTuner로 요청을 보내고 이력과 진행 상태, 보고서를 확인하는 기능을 개발했습니다.
  - 화면별로 SQL과 인스턴스 필드가 달라 각 화면의 행 데이터를 요청 형식으로 바꾸는 규칙을 정의했습니다.
  - 응답을 받지 못해도 요청이 접수됐을 수 있어 실패로 확정하지 않고 '접수 확인 중'으로 표시했습니다.
  - 로컬 MFO와 실제 연동 서버(PJS), ExTuner 모의 응답으로 요청 기록과 접수 실패, 응답 유실 시 동작을 검증했습니다.

Task Analyzer: 작업 맥락 검색 도구

실험 프로젝트

`Python / TypeScript / MCP`

개발에 필요한 요구사항과 관련 코드, 과거 작업 이력을 찾는 검색 도구입니다.

- **여러 도구에서 쓸 수 있는 검색 서버 설계**
  - Task Analyzer 밖에서도 검색 기능을 쓰도록 데이터 관계와 검색 API를 별도 서버로 설계했습니다.
  - 서버는 근거를 검색하고 호출하는 AI 도구는 그 근거로 답변을 만들도록 역할을 나눴습니다.
  - MCP와 스킬에서 같은 API를 호출하도록 검색 입력과 결과 형식을 정했습니다.
- **검색 플러그인을 배포해 회의록 정리와 개발에 사용**
  - Claude Code와 Codex에서 검색 플러그인을 설치하도록 EXEM AI Marketplace를 구성했습니다.
  - 회의록을 정리할 때 관련 태스크를 검색해 링크를 댓글에 남기는 데 쓰고 있습니다.
  - 개발 중에는 관련 이슈와 변경 이력, 문서를 찾아 작업 맥락을 확인하는 데 사용합니다.

GitLab 슬래시 커맨드: 스레드에서 AI와 협업 실험 프로젝트

`TypeScript / LangGraph`

GitLab MR 댓글에서 AI에 코드 질문이나 수정 요청을 보내고 답변을 확인하는 봇입니다.

- **논의 중인 스레드에서 AI 호출과 결과 공유**
  - 개인 PC로 맥락을 옮기지 않고 논의 중인 MR 댓글에서 AI를 호출할 수 있게 했습니다.
  - 답변과 수정 결과를 요청한 스레드에 남겨 동료도 같은 맥락에서 검토할 수 있게 했습니다.
- **스레드별 대화 이력과 MR 공통 요약 분리**
  - MR 안의 서로 다른 논의가 섞이지 않도록 스레드별로 대화 이력을 분리했습니다.
  - 다른 스레드에서도 앞선 작업 결과를 참고하도록 MR 단위로 결과 요약을 공유했습니다.
  - 당시 GitLab에서 두 스레드의 이력은 분리되고 결과 요약은 다음 대화로 전달되는 것을 확인했습니다.

학력

동의대학교	정보통신공학과 / 응용 소프트웨어 공학 부전공 | 2017.03 - 2024.03

최근 글 전체 글 최신 5개 / 2026.10.24 확인

자본주의 1부 - 돈은 빚이다
2026.09.20

지금 테스트 코드가 더더욱 중요한 이유
2026.09.06

스틱 후기
2026.08.29

Tanstack Table V9 마이그레이션 회고
2026.08.26

개발자로 살아남기 - 성장에 대한 고민
2026.08.09